



Ansible主机清单和配置

誉天教育

学习目标

- 定义主机和组
- 选择主机和组
- 配置文件优先级
- 配置文件详解



在使用Ansible来批量管理主机的时候，通常我们需要先定义要管理哪些主机或者主机组，而这个用于管理主机与主机组的文件就叫做Inventory，也叫主机清单。该文件默认位于/etc/ansible/hosts。当然我们也可以通过修改ansible配置文件的Inventory配置项来修改默认inventory的位置。

1. 简单的主机和组

- ◆ 中括号中的名字代表组名，可以根据自己的需求将庞大的主机分成具有标识的组，如上面分了两个组 webservers和dbservers组；
- ◆ 主机(hosts)部分可以使用域名、主机名、IP地址表示；当然使用前两者时，也需要主机能反解析到相应的IP地址，一般此类配置中多使用IP地址；

mail.yanruogu.com

[webservers]

web1.yanruogu.com

web2.yanruogu.com

[dbservers]

db1.yanruogu.com

db2.yanruogu.com

2. 指定主机范围

- 下面指定了从web01到web50，webservers组共计50台主机；databases组有db-a到db-f共6台主机

[webservers]

www[01:50].yanruogu.com

[databases]

db-[a:f].yanruogu.com

3. 定义主机组嵌套

- ◆ 如下示例中，production组包含两个子组，分别为webservers和dbservers，webservers和dbservers组分别包含若干主机

```
[webservers]
```

```
web1.lab.example.com
```

```
web2.lab.example.com
```

```
[dbservers]
```

```
db1.lab.example.com
```

```
db2.lab.example.com
```

```
[production:children]
```

```
webservers
```

```
dbservers
```

```
[production]
```

```
192.168.40.11
```

- 在前面定义Inventory的时候，我们会把所有被管理主机通过主机组的方式定义到Inventory当中，但是当我们实际使用的时候，可能只需要对某一主机或主机组进行操作，这个时候就需要通过匹配的方式指定某一特定主机或主机组。
- 下面先定义一个主机清单示例：

1. 匹配所有主机

ansible all --list-hosts all表示匹配所有的主机

2. 匹配指定的主机或主机组

ansible prod --list-hosts

匹配单个组

ansible db2.example.com --list-hosts

匹配单个主机

ansible 'lb1.lab.example.com,s1.lab.example.com,db1.example.com' --list-hosts

匹配多个主机

ansible 'london,boston' --list-hosts

匹配多个组

ansible ungrouped --list-hosts

匹配不属于任何组的主机

3. 通配符匹配

ansible '*.example.com' --list-hosts 匹配 '*.example.com'

ansible '172.25.*' --list-hosts 匹配 172.25.* 的主机

ansible 's*' --list-hosts 匹配以 s 开头的主机及主机组

4. 通配符组合匹配

ansible '*.example.com,!*.lab.example.com' --list-hosts
匹配包含 *.example.com 但不包含 *.lab.example.com 的主机

ansible 'prod,172*,*lab*' --list-hosts 匹配包含 prod 以及 172 开头、包含 lab 关键字的主机或组

ansible 'db,&london' --list-hosts 匹配属于 db 组同时还属于 london 组的主机

ansible 'boston,london,&prod,!lb' --list-hosts
匹配在 london 组或者 boston 组，还必须在 prod 组中且必须不在 lb 组中的主机

5. 正则表达式匹配

```
# ansible '~(s|db).*\.example\.com' --list-hosts
```

在开头的地方使用“~”，用来表示这是一个正则表达式，以s或者db开头的，中间包含任意长度字符串，并且以example.com结尾的主机或者组。

6. 通过--limit明确指定主机或组

```
# ansible all --limit srv1.example.com --list-hosts
```

 选定的组中明确指定主机

```
# ansible all --limit @retry.txt --list-hosts
```

 指定一个文件，该文件中定义明确指定的主机的列表

定义一个retry.txt如下：

```
srv1.example.com
```

- 配置文件优先级：

ansible的配置文件名为ansible.cfg，它一般会存在于四个地方：

- ◆ ANSIBLE_CONFIG：首先，Ansible命令会检查该环境变量，及这个环境变量将指向的配置文件
- ◆ ./ansible.cfg：当前工作目录，即当前执行ansible指令的目录，如果ANSIBLE_CONFIG环境变量未定义，则优先使用该配置文件
- ◆ ~/.ansible.cfg：当前用户家目录下的一个隐藏文件，如果当前工作目录下不存在ansible.cfg配置文件，则会查找用户家目录下的该隐藏文件
- ◆ /etc/ansible/ansible.cfg：默认配置文件，如果上面两个路径下的ansible.cfg都不存在，则使用该文件

- ansible.cfg的配置默认分为十段：

[defaults]：通用配置项

[inventory]：与主机清单相关的配置项

[privilege_escalation]：特权升级相关的配置项

[paramiko_connection]：使用paramiko连接的相关配置项，Paramiko在RHEL6之后的版本中默认使用的ssh连接方式

[ssh_connection]：使用OpenSSH连接的相关配置项，OpenSSH是Ansible在RHEL6之后默认使用的ssh连接方式

[persistent_connection]：持久连接的配置项

[accelerate]：加速模式配置项

[selinux]：selinux相关的配置项

[colors]：ansible命令输出的颜色相关的配置项

[diff]：定义是否在运行时打印diff（变更前与变更后的差异）

[defaults]

inventory = /etc/ansible/hosts

ask_pass = false

remote_user = root

host_key_checking = False

log_path = /var/log/ansible.log

[privilege_escalation]

become=True

become_method=sudo

become_user=root

become_ask_pass=False

[ssh_connection]

ssh_args = -C -o ControlMaster=auto -o ControlPersist=60s

- ◆ inventory：定义默认使用的主机清单
- ◆ remote_user：ansible在操作远程主机时，使用远程主机上的哪个用户身份，默认是root
- ◆ ask_pass：ansible在操作远程主机时，获取远程主机上的用户身份，是否交互提示密码验证，默认为true。如果使用密钥认证的话，建议将其设置为false
- ◆ log_path：默认ansible 执行的时候，并不会输出日志到文件，打开该配置项，所有的命令执行后，都会将日志输出到/var/log/ansible.log文件。
- ◆ become：如果ansible在操作远程主机时，使用的是远程主机上的普通用户，该普通用户是否允许提权
- ◆ become_method：如果允许提权，使用何种提权方式，默认是sudo
- ◆ become_user：提权到哪个用户身份，默认是root
- ◆ become_ask_pass：提权时，是否交互提示密码验证，默认为False
- ◆ ssh_args：ansible通过ssh连接远程被管理机，这里用于定义一些ssh连接时的参数，如-C启用压缩传输，ControlPersist用于提升性能。
- ◆ host_key_checking：通过ssh首次连接远程主机时，由于在本机的~/.ssh/known_hosts文件中并有fingerprint key串，ssh第一次连接的时候一般会提示输入yes/no进行确认将key字符串加入到~/.ssh/known_hosts文件中。将此项设置为False将跳过该确认过程。或者在ssh_args中添加参数-o StrictHostKeyChecking=no，也可以实现该功能。

- ansible在执行ad-hoc的时候，需要使用远程管理机上的一个用户身份，默认这个用户是root，而在实际生产环境中，不建议直接使用root，而推荐使用一个普通用户，需要配置该普通用户可通过sudo提权。
- 配置步骤如下：
 1. 在被管理机上创建一个普通用户ansible，并配置sudo提权：

```
# useradd ansible 创建ansible用户
```

```
# vim /etc/sudoers.d/ansible设置ansible用户可免密提权
```

```
ansible  ALL=(ALL)  NOPASSWD:ALL
```

2. 在管理端修改ansible.cfg配置文件如下：

```
[default]
```

```
remote_user = ansible
```

```
ask_pass = False
```

```
[privilege_escalation]
```

```
become=True
```

```
become_method=sudo
```

```
become_user=root
```

```
become_ask_pass=False
```

3. ssh的秘钥验证在管理端修改ansible.cfg配置文件如下：

```
# ssh-keygen 生成密钥对
```

```
# ssh-copy-id -i .ssh/id_rsa.pub ansible@db1.example.com 发送公钥到远程主机
```

说明：通过上面的操作，需要为ansible用户先创建密码，在实际生产当中，不建议为该用户设置密码。可在安装系统时，提前将管理端公钥直接写入到被管理节点的ansible用户下。

4. 测试

```
# ansible db1.example.com -m shell -a "whoami"
```

- 定义主机和组
- 选择主机和组
- 配置文件优先级
- 配置文件详解
- 配置案例

